

```

#include <iostream>
#include <iomanip>
#include <cstdlib>
#include <ctime>

using namespace std;

// ---- Function prototypes ----
void generateRandomData(int arr[], int size);
void copyArray(int source[], int dest[], int size);
void bubbleSort(int arr[], int size, long &comparisons, long &swaps);
void insertionSort(int arr[], int size, long &comparisons, long &shifts);
void mergeSort(int arr[], int left, int right, long &comparisons);
void merge(int arr[], int left, int mid, int right, long &comparisons);
void quickSort(int arr[], int low, int high, long &comparisons, long &swaps);
int partition(int arr[], int low, int high, long &comparisons, long &swaps);
bool isSorted(int arr[], int size);
int getValidatedInt(string prompt, int minVal, int maxVal);

int main() {
    srand(time(0));
    cout << fixed << setprecision(6);

    int size = getValidatedInt("How many elements to sort? (100 - 20000): ", 100, 20000);

    int *original = new int[size];
    generateRandomData(original, size);

    cout << "\n===== SORTING ALGORITHM COMPARISON (" << size << " elements)
=====\\n";
    cout << left << setw(16) << "Algorithm" << setw(14) << "Time (s)"
        << setw(16) << "Comparisons" << setw(12) << "Swaps/Shifts" << endl;
    cout << string(58, '-') << endl;

    // ---- Bubble Sort ----
    {
        int *data = new int[size];
        copyArray(original, data, size);
        long comparisons = 0, swaps = 0;
        clock_t start = clock();
        bubbleSort(data, size, comparisons, swaps);
        clock_t end = clock();
        double elapsed = double(end - start) / CLOCKS_PER_SEC;
        cout << left << setw(16) << "Bubble Sort" << setw(14) << elapsed
            << setw(16) << comparisons << setw(12) << swaps << endl;
        delete[] data;
    }

    // ---- Insertion Sort ----
    {
        int *data = new int[size];
        copyArray(original, data, size);
        long comparisons = 0, shifts = 0;
        clock_t start = clock();

```

```

        insertionSort(data, size, comparisons, shifts);
        clock_t end = clock();
        double elapsed = double(end - start) / CLOCKS_PER_SEC;
        cout << left << setw(16) << "Insertion Sort" << setw(14) << elapsed
            << setw(16) << comparisons << setw(12) << shifts << endl;
        delete[] data;
    }

    // ---- Merge Sort ----
    {
        int *data = new int[size];
        copyArray(original, data, size);
        long comparisons = 0;
        clock_t start = clock();
        mergeSort(data, 0, size - 1, comparisons);
        clock_t end = clock();
        double elapsed = double(end - start) / CLOCKS_PER_SEC;
        cout << left << setw(16) << "Merge Sort" << setw(14) << elapsed
            << setw(16) << comparisons << setw(12) << "-" << endl;
        if (!isSorted(data, size)) cout << " WARNING: merge sort output not sorted!\n";
        delete[] data;
    }

    // ---- Quick Sort ----
    {
        int *data = new int[size];
        copyArray(original, data, size);
        long comparisons = 0, swaps = 0;
        clock_t start = clock();
        quickSort(data, 0, size - 1, comparisons, swaps);
        clock_t end = clock();
        double elapsed = double(end - start) / CLOCKS_PER_SEC;
        cout << left << setw(16) << "Quick Sort" << setw(14) << elapsed
            << setw(16) << comparisons << setw(12) << swaps << endl;
        if (!isSorted(data, size)) cout << " WARNING: quick sort output not sorted!\n";
        delete[] data;
    }

    cout << "\nNote: clock() measures CPU time, which can read as 0.000000 for\n"
        << "small sizes on fast machines. Try a larger element count (e.g.\n"
        << "10000-20000) to see clearer timing differences.\n";

    delete[] original;
    return 0;
}

// Fills arr with random ints between 0 and 99999
void generateRandomData(int arr[], int size) {
    for (int i = 0; i < size; i++) arr[i] = rand() % 100000;
}

// Copies size elements from source into dest
void copyArray(int source[], int dest[], int size) {
    for (int i = 0; i < size; i++) dest[i] = source[i];
}

```

```
}
```

```
// Bubble sort with early-exit optimization
```

```
void bubbleSort(int arr[], int size, long &comparisons, long &swaps) {  
    for (int i = 0; i < size - 1; i++) {  
        bool swapped = false;  
        for (int j = 0; j < size - i - 1; j++) {  
            comparisons++;  
            if (arr[j] > arr[j + 1]) {  
                int temp = arr[j];  
                arr[j] = arr[j + 1];  
                arr[j + 1] = temp;  
                swaps++;  
                swapped = true;  
            }  
        }  
        if (!swapped) break;  
    }  
}
```

```
// Insertion sort
```

```
void insertionSort(int arr[], int size, long &comparisons, long &shifts) {  
    for (int i = 1; i < size; i++) {  
        int key = arr[i];  
        int j = i - 1;  
        while (j >= 0) {  
            comparisons++;  
            if (arr[j] > key) {  
                arr[j + 1] = arr[j];  
                shifts++;  
                j--;  
            } else {  
                break;  
            }  
        }  
        arr[j + 1] = key;  
    }  
}
```

```
// Merge sort (divide step)
```

```
void mergeSort(int arr[], int left, int right, long &comparisons) {  
    if (left >= right) return;  
    int mid = left + (right - left) / 2;  
    mergeSort(arr, left, mid, comparisons);  
    mergeSort(arr, mid + 1, right, comparisons);  
    merge(arr, left, mid, right, comparisons);  
}
```

```
// Merge sort (combine step) - uses temporary plain arrays, not vectors
```

```
void merge(int arr[], int left, int mid, int right, long &comparisons) {  
    int leftSize = mid - left + 1;  
    int rightSize = right - mid;  
  
    int *leftPart = new int[leftSize];
```

```

int *rightPart = new int[rightSize];

for (int i = 0; i < leftSize; i++) leftPart[i] = arr[left + i];
for (int j = 0; j < rightSize; j++) rightPart[j] = arr[mid + 1 + j];

int i = 0, j = 0, k = left;
while (i < leftSize && j < rightSize) {
    comparisons++;
    if (leftPart[i] <= rightPart[j]) {
        arr[k++] = leftPart[i++];
    } else {
        arr[k++] = rightPart[j++];
    }
}
while (i < leftSize) arr[k++] = leftPart[i++];
while (j < rightSize) arr[k++] = rightPart[j++];

delete[] leftPart;
delete[] rightPart;
}

// Quick sort (Lomuto partition scheme, last element as pivot)
void quickSort(int arr[], int low, int high, long &comparisons, long &swaps) {
    if (low < high) {
        int pivotIndex = partition(arr, low, high, comparisons, swaps);
        quickSort(arr, low, pivotIndex - 1, comparisons, swaps);
        quickSort(arr, pivotIndex + 1, high, comparisons, swaps);
    }
}

int partition(int arr[], int low, int high, long &comparisons, long &swaps) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        comparisons++;
        if (arr[j] < pivot) {
            i++;
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
            swaps++;
        }
    }
    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;
    swaps++;
    return i + 1;
}

bool isSorted(int arr[], int size) {
    for (int i = 1; i < size; i++)
        if (arr[i - 1] > arr[i]) return false;
    return true;
}

```

```
}  
  
int getValidatedInt(string prompt, int minVal, int maxVal) {  
    int value;  
    while (true) {  
        cout << prompt;  
        if (cin >> value && value >= minVal && value <= maxVal) {  
            cin.ignore(1000, '\n');  
            return value;  
        }  
        cout << "Error: Enter a value between " << minVal << " and " << maxVal << ".\n";  
        cin.clear();  
        cin.ignore(1000, '\n');  
    }  
}
```